

UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software
Equipe: José Mykael Alves Nogueira e Davi Sousa Facundo
Projeto: Dask
Fork: https://github.com/dsfacundo/dask/tree/refatoracao-test-smells

1. test_dataframe.py — Conditional Test Logic (Linha 437)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método de teste `test_describe` do arquivo `dask/dataframe/tests/test_dataframe.py` (originalmente na linha 437: `if subset is not None:`).
Minhas principais dificuldades na remoção do code smell são: O teste original misturava dois cenários distintos na mesma função parametrizada: (1) testes com filtragem de subconjunto de colunas (`subset is not None`), que validavam apenas o `DataFrame.describe()`, e (2) testes sem `subset` (`subset is None`), que validavam tanto o `DataFrame.describe()` quanto um laço `for` sobre colunas individuais (`Series.describe()`), além de ramificações para remoção de média em colunas de timestamp e compatibilidade com `PANDAS_GE_310`.
Usei o CATSPY para: Identificação e localização do smell *Conditional Test Logic* na linha 437, compreensão do critério de detecção (estruturas condicionais `if` e laços `for` no corpo do teste) e verificação pós-refatoração.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Split Test (Divisão de Teste), Parameterized Test (`@pytest.mark.parametrize`) e Extract Method (Extração de Método Auxiliar). Dividi o teste em `test_describe_subset` (exclusivo para subconjuntos de colunas) e `test_describe` (parametrizado também por coluna `col`), extraindo a criação de dados e normalização do resultado esperado para as funções auxiliares `_make_describe_df`, `_describe_expected_df` e `_describe_series_pair`.

2. test_groupby.py — Conditional Test Logic (Linha 1123)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método de teste `test_dataframe_aggregations_multilevel` do arquivo `dask/dataframe/tests/test_groupby.py` (originalmente na linha 1123: `if agg_func not in ("cov", "corr"):`).

Minhas principais dificuldades na remoção do code smell são: O parâmetro `agg_func` vem de uma fixture global da suíte de testes e inclui funções com restrições matemáticas específicas no Pandas/Dask (ex.: `cov` e `corr` exigem múltiplas colunas e ordenação especial de índices/colunas; `nunique` não é suportado em certas seleções de múltiplas colunas). Remover os `if` sem quebrar a combinação com a fixture exigiu isolar a lógica de verificação.
Usei o CATSPY para: Detecção do Conditional Test Logic na linha 1123 e análise das ramificações internas do método de teste
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method (Extração de Método Auxiliar / Custom Assertion Helper). Extraí a lógica de verificação específica de cada função de agregação para o método auxiliar `_check_multilevel_agg`, deixando o corpo de `test_dataframe_aggregations_multilevel` linear, focado apenas no Arrange dos DataFrames e na chamada de verificação.

3. test_shuffle.py — Verbose Test (Linhas 415–490)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test no método `test_set_index_sorts` do arquivo `dask/dataframe/tests/test_shuffle.py` (originalmente entre as linhas 415 e 490, totalizando 76 linhas).
Minhas principais dificuldades na remoção do code smell são: O teste reproduz uma regressão real (issue #2288) que depende de uma sequência específica de 58 timestamps em nanossegundos divididos em 3 partições específicas (`breaks = [10, 36, 58]`). A dificuldade foi reduzir drasticamente o tamanho do teste sem alterar nenhum valor do conjunto de dados que reproduz o bug de ordenação.
Usei o CATSPY para: Identificação do smell Verbose Test (delimitado entre as linhas 415 e 490) devido ao excesso de linhas ocupadas pela declaração literal de dados e montagem de partições dentro do método de teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method (Extração de Helper de Fixture/Setup) e Compactação de Dados de Setup. Extraí a construção do `DataFrame` Dask particionado para a função auxiliar `_make_unsorted_timestamp_ddf`, reduzindo `test_set_index_sorts` de 76 linhas para apenas 4 linhas claras e diretas.

4. test_ufunc.py — Verbose Test (Linhas 92–150)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test no método `test_ufunc` do arquivo `dask/dataframe/tests/test_ufunc.py` (originalmente das linhas 92 a 150, com 59 linhas).
Minhas principais dificuldades na remoção do code smell são: O teste validava duas responsabilidades distintas na mesma função: (1) aplicação de ufuncs do Dask/NumPy sobre `Series` e `DataFrame`, e (2) aplicação sobre `Index`, usando retornos antecipados (`if ...: return`) quando o índice era `str`/`object` ou quando a ufunc não suportava `Index`.
Usei o CATSPY para: Detectar que o método `test_ufunc` excedia o limite de extensão (Verbose Test, linhas 92–150) por acumular múltiplos alvos de teste e verificações condicionais.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Split Test (Divisão de Teste), Extract Method e Filtragem na Parametrização. Extraí o helper `_expected_ufunc_type` e separei os testes de `Index` em um novo teste parametrizado dedicado `test_ufunc_index` (recebendo apenas entradas com índice numérico e a lista filtrada `_INDEX_UFUNCS`), deixando ambos os testes curtos (~20 linhas) e sem desvios condicionais.

5. test_utils_dataframe.py — Verbose Test (Linhas 286–359)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test no método `test_meta_nonempty_index` do arquivo `dask/dataframe/tests/test_utils_dataframe.py` (originalmente das linhas 286 a 359, com 74 linhas).
Minhas principais dificuldades na remoção do code smell são: O método testava sequencialmente 10 tipos/configurações diferentes de índices do Pandas (`RangeIndex`, `Index` inteiro/objeto, `DatetimeIndex`, `PeriodIndex`, `TimedeltaIndex`, `CategoricalIndex` conhecido/desconhecido e dois `MultiIndex`), cada qual verificando um subconjunto ligeiramente diferente de atributos (`tz`, `freq`, `dtype`, `ordered`, `categories`, `levels`).
Usei o CATSPY para: Localizar o Verbose Test (linhas 286–359) causado pela repetição sequencial de blocos Arrange-Act-Assert quase idênticos dentro de um único teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Parameterized Test (`@pytest.mark.parametrize`) e Split Test. Parametrizei os 8 casos de índices simples em `test_meta_nonempty_index(idx, expected_attrs)` (com apenas 6 linhas no corpo), separei a asserção específica de

`categories` em `test_meta_nonempty_categorical_index_categories` e parametrizei os casos multinível em `test_meta_nonempty_multiindex`.

6. test_resample_expr.py — Conditional Test Logic (Linha 60)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método `test_resample_apis` do arquivo `dask/dataframe/tseries/tests/test_resample_expr.py` (originalmente na linha 60: `if api != "size":` e linha 65: `if api != "ohlc":`).
Minhas principais dificuldades na remoção do code smell são: A lista parametrizada de 16 métodos de resample incluía `"size"` (que retorna uma `Series` sem colunas indexáveis por `"foo"`) e `"ohlc"` (que retorna um `DataFrame` mesmo quando aplicado sobre uma coluna, impedindo a comparação direta de `.simplify()`).
Usei o CATSPY para: Identificar o Conditional Test Logic na linha 60 e constatar que o teste utilizava `if` aninhados para filtrar parâmetros incompatíveis em tempo de execução.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Split Test (Divisão de Teste) combinado com Parameterized Test. Defini a constante `_RESAMPLE_SERIES_APIS` e dividi o teste em três funções parametrizadas sem nenhum `if`: `test_resample_apis` (todas as APIs), `test_resample_apis_column_selection` (excluindo `"size"`) e `test_resample_apis_simplify` (excluindo `"size"` e `"ohlc"`).

7. test_progress.py — Non-Functional Statement (Linha 59)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non-Functional Statement no método `test_clean_exit` do arquivo `dask/diagnostics/tests/test_progress.py` (originalmente na linha 59: instrução `pass` dentro de `except ZeroDivisionError:`).
Minhas principais dificuldades na remoção do code smell são: Garantir que a exceção `ZeroDivisionError` provocada pela tarefa `{"a": (lambda: 1 / 0,)}` continue sendo capturada exatamente no escopo externo ao context manager `with ProgressBar() as pbar:`, de modo que a variável `pbar` permaneça acessível para as asserções de limpeza (`assert not pbar._running` e `assert not pbar._timer.is_alive()`).
Usei o CATSPY para:

Localizar a instrução não funcional (`pass`) na linha 59 e identificar o padrão inadequado de captura manual de exceção com bloco vazio.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Try-Except-Pass with `pytest.raises`. Substituí o bloco `try ... except ZeroDivisionError: pass` pelo gerenciador de contexto nativo do Pytest `with pytest.raises(ZeroDivisionError):`, eliminando a instrução `pass` e garantindo que o teste falhe caso a exceção esperada não seja lançada.

8. test_callbacks.py — Exception Handling (Linha 50)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling no método `test_finish_always_called` do arquivo `dask/tests/test_callbacks.py` (originalmente na linha 50 e linha 59: blocos `try ... except Exception as e: assert isinstance(e, ZeroDivisionError)`).
Minhas principais dificuldades na remoção do code smell são: O método original usava `try/except` manual nas duas primeiras verificações (`get_sync` e `get_threaded`), o que permitia que o teste passasse silenciosamente caso nenhuma exceção fosse levantada! Era necessário padronizar o tratamento com o restante do teste mantendo a verificação de que `flag[0]` foi atualizado pelo callback `_finish`.
Usei o CATSPY para: Detectar o smell Exception Handling na linha 50 causado pelo uso explícito de blocos `try/except` dentro do método de teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Exception Handling with Expected Exception (`pytest.raises`). Substituí ambos os blocos `try ... except Exception as e:` por `with pytest.raises(ZeroDivisionError):`, tornando o teste mais expressivo, seguro e livre de tratamento manual de exceções.

9. test_cli.py — Conditional Test Logic (Linha 88)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método `test_config_set_value` do arquivo `dask/tests/test_cli.py` (originalmente na linha 88: `if not empty_config:`, além das linhas 99 e 104).
Minhas principais dificuldades na remoção do code smell são: O teste possui uma matriz combinatória de 4 parâmetros (`value`, `empty_config`, `file`,

`existing_key_config_file`) que alteram a preparação do diretório temporário de configuração e os argumentos passados ao `CliRunner`.
Usei o CATSPY para: Identificar o Conditional Test Logic na linha 88 e mapear os blocos `if/else` de configuração de cenário dentro do método de teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method (Extração de Helper de Setup). Extraí a lógica de preparação do cenário (criação de arquivos YAML prévios e montagem dos argumentos da CLI) para a função auxiliar `_setup_config_set_scenario`, deixando `test_config_set_value` inteiramente sequencial (Arrange → Act → Assert) e sem condicionais.

10. test_distributed.py — Conditional Test Logic (Linha 500)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método `test_blockwise_array_creation` do arquivo `dask/tests/test_distributed.py` (originalmente na linha 500: cadeia `if io == "ones": ... elif io == "zeros": ... elif io == "full": ...`).
Minhas principais dificuldades na remoção do code smell são: As funções `ones` e `zeros` recebem apenas `shape`, enquanto `full` exige um argumento posicional adicional (`fill_value = 10`), o que levou o autor original a usar um `if/elif` dentro do teste para instanciar `darr` e `narr`.
Usei o CATSPY para: Detectar o Conditional Test Logic na linha 500 decorrente do despacho condicional baseado em string dentro do teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Parameterized Test Enhancement (Enriquecimento de Parametrização com Despacho Dinâmico). Alterei `@pytest.mark.parametrize("io,extra_args", [("ones", ()), ("zeros", ()), ("full", (10,))])` e instanciei os arrays diretamente com `getattr(da, io)(shape, *extra_args, chunks=chunks)` e `getattr(np, io)(shape, *extra_args)`, eliminando completamente a cadeia `if/elif`.

11. test_distributed.py — Verbose Test (Linhas 968–1027)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test no método `test_get_scheduler_default_client_config_interleaving` do arquivo `dask/tests/test_distributed.py` (originalmente das linhas 968 a 1027, com 60 linhas).

Minhas principais dificuldades na remoção do code smell são: O teste verifica o entrelaçamento de contextos (`dask.config.set` e `Client(...,set_as_default=...)` / `client.as_current()`) e precisa garantir o fechamento explícito de cada `Client` com `client.close()` para não vazar conexões para o cluster s.
Usei o CATSPY para: Identificar o smell Verbose Test (linhas 968–1027) causado pelo encadeamento excessivo de cenários de configuração de escalonador em uma única função.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method e Split Test. Criei o método auxiliar `_assert_client_scheduler` para encapsular o ciclo de criação/asserção/fechamento básico do `Client` e dividi os cenários em dois testes coesos e curtos (~20 linhas cada): `test_get_scheduler_default_client_config_interleaving` e `test_get_scheduler_client_race_and_config_override`.

12. test_hlgexpr.py — Verbose Test (Linhas 63–134)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test no método `test_hlg_expr_sequence_finalize` do arquivo `dask/tests/test_hlgexpr.py` (originalmente das linhas 63 a 134, com 72 linhas).
Minhas principais dificuldades na remoção do code smell são: Mais da metade da função original (30 linhas) consistia na montagem manual de três `HighLevelGraph` (`hlgx`, `hlgy`, `hlgz`) e três `HLGExpr` (`hlgexprx`, `hlgexpry`, `hlgexprz`), seguida tanto por asserções de expressões individuais quanto de sequências (`_ExprSequence`).
Usei o CATSPY para: Detectar o Verbose Test (linhas 63–134) e observar a sobrecarga de código de configuração de grafos dentro da função de teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method (Extração de Fixture Helper) e Split Test. Extraí a construção dos grafos e expressões para `_make_sample_hlg_exprs` e dividi as asserções em `test_hlg_expr_single_finalize` (otimização de expressões individuais) e `test_hlg_expr_sequence_finalize` (otimização de `_ExprSequence`).

13. test_fft.py — Exception Handling (Linha 182)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling no método `test_wrap_fftns` do arquivo

`dask/array/tests/test_fft.py` (originalmente na linha 182: bloco `try: func = getattr(fft_mod, funcname) except AttributeError: pytest.skip(...)`).
Minhas principais dificuldades na remoção do code smell são: Se substituíssemos o `try/except AttributeError` diretamente por um `if not hasattr(...)` dentro do corpo do teste, eliminaríamos o smell Exception Handling, porém introduziríamos um novo smell (Conditional Test Logic) na mesma função!
Usei o CATSPY para: Identificar o Exception Handling na linha 182 e garantir que a solução escolhida não introduzisse outro test smell detectado pela ferramenta.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method (Extração de Helper de Resolução de Módulo/Função) e uso de `getattr(..., None)`. Criei a função auxiliar `_get_fft_func(modname, funcname)` fora do teste, que importa o módulo via `pytest.importorskip` e obtém a função ou aciona `pytest.skip`, mantendo `test_wrap_fftns` livre de `try/except` e de `if`.

14. test_percentiles.py — Exception Handling (Linha 64)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling no método `test_percentile_with_categoricals` do arquivo `dask/array/tests/test_percentiles.py` (originalmente na linha 64: `try: import pandas as pd except ImportError: return`).
Minhas principais dificuldades na remoção do code smell são: O uso de `try: import ... except ImportError: return` não apenas caracteriza Exception Handling no teste, como também faz o teste ser reportado como PASSED (falso positivo) em vez de SKIPPED quando a dependência opcional `pandas` não está instalada.
Usei o CATSPY para: Detectar o bloco `try/except ImportError` na linha 64 como Exception Handling.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Conditional Import with `pytest.importorskip`. Substituí o bloco `try ... except ImportError: return` de 4 linhas por `pd = pytest.importorskip("pandas")`, padrão idiomático do Pytest e do Dask.

15. test_routines.py — Exception Handling (Linha 269)

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Exception Handling no método `test_rot90` do arquivo `dask/array/tests/test_routines.py` (originalmente na linha 269: bloco `try: for axis in axes[:2]: range(np_a.ndim)[axis] except IndexError: ...`).
Minhas principais dificuldades na remoção do code smell são: O teste original fazia o produto cartesiano entre entradas válidas e inválidas (`shape` de 0D a 3D vs. `axes` válidos e inválidos) e usava `try/except IndexError` junto com `if/else` em tempo de execução para decidir se o teste deveria esperar `ValueError` ou comparar `da.rot90` com `np.rot90`.
Usei o CATSPY para: Identificar o Exception Handling na linha 269 (além das condicionais e laços aninhados no bloco `else`) e guiar a separação entre casos de sucesso e casos de erro.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Split Test (Separação de Caminhos Feliz e de Exceção) e Parameterized Test. Dividi o teste em `test_rot90` (parametrizado apenas com combinações válidas de `kwargs`, `shape` e `k`) e `test_rot90_invalid` (parametrizado com as combinações inválidas e verificando `with pytest.raises(ValueError):`), eliminando o `try/except` e todos os `if/for` do teste.

16. test_xarray.py — Exception Handling (Linha 39)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling no método `test_asarray_xarray_intersphinx_workaround` do arquivo `dask/array/tests/test_xarray.py` (originalmente na linha 39: bloco `try: xr.DataArray.__module__ = "xarray" ... finally: xr.DataArray.__module__ = module`).
Minhas principais dificuldades na remoção do code smell são: O teste modifica um atributo global da classe `xr.DataArray` (`__module__`) para simular o comportamento do intersphinx e precisa garantir que o valor original seja restaurado mesmo se uma asserção falhar, para não poluir os demais testes da suíte.
Usei o CATSPY para: Localizar o smell Exception Handling na linha 39 causado pelo uso manual de `try/finally` para teardown de estado global dentro do teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Try-Finally Teardown with Fixture (`monkeypatch`). Injetei a fixture nativa `monkeypatch` do Pytest no teste e utilizei `monkeypatch.setattr(xr.DataArray, "__module__", "xarray")`, que restaura automaticamente o atributo original ao final do teste sem necessidade de `try/finally`.

17. test_base.py — Sleepy Test (Linha 936)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Sleepy Test no método `test_num_workers_config` do arquivo `dask/tests/test_base.py` (originalmente na linha 936: `f = delayed(pure=False)(time.sleep)`).
Minhas principais dificuldades na remoção do code smell são: O teste verifica se o escalonador utiliza simultaneamente todos os `num_workers = 3` configurados (tanto para `"threads"` quanto para `"processes"`, o que exige que a função agendada seja serializável via pickle no Windows e mantenha cada worker ocupado tempo suficiente para que o `Profiler` registre os 3 `worker_ids` distintos).
Usei o CATSPY para: Detectar o smell Sleepy Test na linha 936 devido à referência direta a `time.sleep` dentro do método de teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method (Extração de Função de Tarefa de Worker). Substituí o uso direto de `time.sleep` no teste pela função auxiliar de nível de módulo `_hold_worker_slot(duration)` baseada em `time.perf_counter()` (totalmente compatível com `threads` e `processes` via pickle), removendo a chamada a `sleep` e a compreensão de lista de dentro do teste.

18. test_overlap.py — Redundant Print (Linha 599)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Redundant Print no método `test_map_overlap_trim_false_chunking` do arquivo `dask/array/tests/test_overlap.py` (originalmente nas linhas 599–600: `print(d.shape)` e `print(d.compute().shape)`).
Minhas principais dificuldades na remoção do code smell são: Ao analisar o teste, percebi que ele não possuía nenhuma instrução `assert`: o autor original havia deixado dois `print()` para inspecionar manualmente se o formato (`shape`) estimado do grafo Dask coincidia com o formato real após `.compute()`. Apenas apagar os `print()` deixaria o teste sem asserção (Assertionless Test).
Usei o CATSPY para: Identificar o smell Redundant Print na linha 599 e inspecionar o propósito das impressões em tela no final do teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Print with Assertion. Substituí os dois comandos `print(d.shape)` e `print(d.compute().shape)` pela asserção explícita `assert d.shape == d.compute().shape`, removendo o Redundant Print e garantindo a verificação

automatizada do comportamento esperado.

19. test_shuffle.py (dask_expr) — Redundant Print (Linha 549)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Redundant Print no método `test_set_index_head_nlargest_string` do arquivo `dask/dataframe/dask_expr/tests/test_shuffle.py` (originalmente na linha 549: `print(df.dtypes)`).
Minhas principais dificuldades na remoção do code smell são: Verificar se `print(df.dtypes)` tinha algum efeito colateral relevante (como forçar avaliação de metadados) antes de removê-lo. Como `from_pandas` já inicializa `_meta` imediatamente e as linhas seguintes já executam `assert_eq`, o `print` era puramente resquício de depuração.
Usei o CATSPY para: Localizar a instrução `print(df.dtypes)` na linha 549 classificada como Redundant Print.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Remove Redundant Print (Remoção de Instrução de Depuração). Removi a chamada `print(df.dtypes)` do corpo de `test_set_index_head_nlargest_string`, mantendo as asserções `assert_eq` intactas.

20. test_threaded.py — Conditional Test Logic (Linha 101)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método `test_threaded_within_thread` do arquivo `dask/tests/test_threaded.py` (originalmente na linha 101: laço `while threading.active_count() > before + 10:`, além do laço `for _ in range(20):` na linha 92).
Minhas principais dificuldades na remoção do code smell são: O teste cria 20 threads daemon que invocam o escalonador `threaded.get` internamente e depois aguarda a finalização das threads do pool com um polling controlado por timeout de 5 segundos. Era preciso extrair a mecânica de repetição e espera sem alterar a semântica de concorrência do teste.
Usei o CATSPY para: Detectar o Conditional Test Logic na linha 101 (laço `while` de sincronização) e o laço `for` de criação de threads dentro do método de teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method (Extração de Helpers de Execução e Sincronização). Extraí o ciclo de

criação/junção das threads para `_spawn_and_join_worker_threads` e a espera ativa com timeout para `_wait_for_thread_cleanup`, deixando `test_threaded_within_thread` enxuto e livre de estruturas de repetição/condicionais.

21. test_array_core.py — Verbose Test (Linhas 2089–2140)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test no método `test_store_delayed_target` do arquivo `test_array_core.py` (originalmente das linhas 2089 a 2140, com 52 linhas).
Minhas principais dificuldades na remoção do code smell são: O teste misturava quatro responsabilidades diferentes em uma única função longa: (1) armazenamento sem retorno (`return_stored=False`), (2) um laço `for st_compute in [False, True]:` limpando o dicionário compartilhado `targs.clear()`, (3) uma condicional `if st_compute:` verificando persistência dos dados, e (4) três verificações de erro `ValueError` usando `pytest.raises` na forma legada com lambda ao final do laço.
Usei o CATSPY para: Identificar o smell Verbose Test (linhas 2089–2140), além das estruturas condicionais e laços internos (`for` e `if` nas linhas 2118–2122), e validar sua remoção após a divisão dos cenários.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Fixture (`@pytest.fixture`), Split Test (Divisão de Teste) e Parameterized Test (`@pytest.mark.parametrize`). Extraí a criação dos arrays e alvos delayed para a fixture `delayed_targets` e dividi o teste original em quatro testes curtos, independentes e sem desvios condicionais: `test_store_delayed_target`, `test_store_delayed_target_return_stored` (parametrizado com `st_compute`), `test_store_delayed_target_persisted` e `test_store_delayed_target_invalid_inputs` (utilizando `with pytest.raises(ValueError):`).

22. test_cupy_linalg.py — Verbose Test (Linhas 194–262)

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Verbose Test no método `test_tsqr_uncertain` do arquivo `test_cupy_linalg.py` (originalmente das linhas 194 a 262, com 69 linhas).
Minhas principais dificuldades na remoção do code smell são: O teste validava matrizes CuPy com dimensões de chunks incertas (uncertain chunks geradas por indexação booleana `vary_rows` e `vary_cols`) e acumulava no mesmo corpo: a geração estocástica da matriz na GPU, o teste completo de decomposição QR, o teste completo de decomposição SVD (`compute_svd=True`) e um ramo `else:` para casos inválidos (`error_type is not None`).
Usei o CATSPY para: Detectar o smell Verbose Test entre as linhas 194 e 262 (bem como os desvios condicionais internos nas linhas 203, 207 e 226) causado pelo acúmulo de setup complexo e três fluxos distintos de verificação na mesma função.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Method (Extração de Helper de Setup) e Split Test (Divisão por Responsabilidade e Caminho de Exceção). Extraí a construção dos arrays com dimensões incertas para `_make_uncertain_data` e dividi o método em três testes enxutos e focados: `test_tsqr_uncertain_qr` (validação da fatoração QR), `test_tsqr_uncertain_svd` (validação de valores singulares e ortonormalidade no SVD) e `test_tsqr_uncertain_error` (validação das exceções esperadas com `pytest.raises`).

23. test_cupy_core.py — Verbose Test (Linhas 578–663)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test no método `test_setitem_errs` do arquivo `test_cupy_core.py` (originalmente das linhas 578 a 663, com 86 linhas).
Minhas principais dificuldades na remoção do code smell são: O método enfileirava mais de 20 blocos sequenciais de `with pytest.raises(...):` testando diferentes falhas de atribuição indexada (`__setitem__`) em arrays CuPy/Dask — misturando incompatibilidade de formato (`ValueError`), extrapolação de dimensões/índices escalares (`IndexError`) e indexações múltiplas não suportadas (`NotImplementedError`), além de casos com máscaras booleanas dask.array e broadcasting.
Usei o CATSPY para: Localizar o smell Verbose Test (linhas 578–663) provocado pela longa sequência repetitiva de blocos de verificação de erro em um único método monolítico.
Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Split Test (Agrupamento por Categoria de Exceção) e Parameterized Test (`@pytest.mark.parametrize`). Organizei e separei os cenários de falha em testes coesos e parametrizados por `(index, value)`: `test_setitem_shape_mismatch_errs`, `test_setitem_boolean_and_broadcast_value_errs`, `test_setitem_index_errs`, `test_setitem_scalar_index_err` e `test_setitem_not_implemented_errs`.

24. test_base.py — Verbose Test (Linhas 106–167)

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Verbose Test no método `test_unpack_collections` do arquivo `test_base.py` (originalmente das linhas 106 a 167, com 62 linhas).

Minhas principais dificuldades na remoção do code smell são:
O teste declarava classes auxiliares (`ADataClass`, `ANamedTuple`) e uma função aninhada `build(...)` com estruturas profundamente aninhadas (tuplas, dicionários com coleções como chave, listas, set, OrderedDict, dataclass, NamedTuple e geradores) dentro do próprio corpo do teste, e em seguida verificava quatro comportamentos distintos (`traverse=True`, `traverse=False`, entrada sem coleções e literais que imitam tarefas Dask).

Usei o CATSPY para:
Identificar o Verbose Test delimitado entre as linhas 106 e 167 decorrente da definição interna de tipos/estruturas de dados e da verificação sequencial de múltiplos modos de unpack_collections.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Extract Method / Move Helper Classes to Module Level e Split Test. Movi `ADataClass`, `ANamedTuple` e a função construtora `_build_test_structure` para o nível do módulo e dividi os cenários em quatro testes curtos e autoexplicativos: `test_unpack_collections_standard`, `test_unpack_collections_no_traverse`, `test_unpack_collections_empty` e `test_unpack_collections_task_literals`.

25. test_task_spec.py — Verbose Test (Linhas 861–907)

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Verbose Test no método `test_linear_fusion` do arquivo `test_task_spec.py` (originalmente das linhas 861 a 907, com 47 linhas).

Minhas principais dificuldades na remoção do code smell são: O teste original mutava incrementalmente a mesma lista `tasks` (`tasks.append(...)`) ao longo da função para testar quatro cenários progressivos de fusão linear de grafos (`fuse_linear_task_spec`): cadeia simples de Task, cadeia iniciada por DataNode, ramificação parcial após o terceiro nó e ramificação dupla impedindo qualquer fusão.
Usei o CATSPY para: Detectar o Verbose Test nas linhas 861–907 causado pelo reaproveitamento mutável de estado entre múltiplos cenários de fusão dentro da mesma função de teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Fixture (`task_chain` e `datanode_chain`), Extract Method (`fuse` e Custom Assertion `assert_is_fused_subgraph`) e Split Test. Dividi a função original em quatro testes independentes e sem mutação compartilhada: `test_linear_chain_is_fused_into_single_subgraph`, `test_data_nodes_are_not_fused`, `test_branch_after_third_stops_fusion_at_branch_point` e `test_branches_after_second_and_third_prevent_any_fusion`.

26. test_task_spec.py — Verbose Test (Linhas 970–1031)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Verbose Test no método `test_dict_class` do arquivo `test_task_spec.py` (originalmente das linhas 970 a 1031, com 62 linhas).
Minhas principais dificuldades na remoção do code smell são: O método testava toda a API da classe Dict do task_spec de uma só vez, reatribuindo repetidamente as variáveis `t` e d para validar resolução de TaskRef, chaves hashable não-string, igualdade e tokenize independentes da ordem dos kwargs, construção por lista de pares, protocolo Mapping, len() e desempacotamento via **kwargs.
Usei o CATSPY para: Identificar o smell Verbose Test (linhas 970–1031) gerado pelo acúmulo de mais de dez verificações comportamentais independentes da classe Dict em um único teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Extract Fixture (`simple_dict`), Split Test e Parameterized Test (`@pytest.mark.parametrize`). Extraí a instância base para a fixture `simple_dict`, parametrizei os formatos de pares (tuple-pairs e list-pairs) em `test_dict_from_pairs_keeps_plain_list_values` e separei cada comportamento da classe Dict em testes unitários atômicos e bem nomeados (como `test_dict_with_task_ref_exposes_dependencies_and_resolves_them`, `test_dict_equality_and_token_ignore_kwargs_order` e `test_dict_can_be_unpacked_as_kwargs`).

27. test_bag.py — Conditional Test Logic (Linha 1249)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método `test_range` do arquivo `test_bag.py` (originalmente na linha 1249: laço `for npartitions in [1, 7, 10, 28]:`).
Minhas principais dificuldades na remoção do code smell são: O uso de um laço `for` manual dentro do teste faz com que todas as configurações de partições sejam executadas como um único caso de teste (interrompendo a validação das demais caso a primeira falhe e dificultando o diagnóstico), além de introduzir lógica de controle de fluxo no corpo do teste.
Usei o CATSPY para: Detectar o smell Conditional Test Logic na linha 1249 causado pela iteração explícita `for` sobre os valores de entrada dentro do método `test_range`.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Parameterized Test (`@pytest.mark.parametrize`). Substituí o laço `for npartitions in [1, 7, 10, 28]:` pelo decorador `@pytest.mark.parametrize("npartitions", [1, 7, 10, 28])`, alinhando `test_range` ao padrão já utilizado no teste vizinho `test_zip` e deixando o corpo puramente sequencial.

28. test_order.py — Conditional Test Logic (Linha 334)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método `test_base_of_reduce_preferred` do arquivo `test_order.py` (originalmente na linha 334: `if data_root: dsk[c] = 1 else: dsk[c] = (f, 1)`, além das compreensões de dicionário com `for` nas linhas 331 e 333).
Minhas principais dificuldades na remoção do code smell são: O teste recebia uma flag booleana `data_root` apenas para decidir via `if/else` qual valor atribuir ao nó raiz c do grafo DAG (1 para nó de dados puro ou (f, 1) para tarefa computável), além de montar o pequeno grafo fixo de 9 nós por meio de dict comprehensions.
Usei o CATSPY para: Localizar o Conditional Test Logic na linha 334 (e as compreensões com `for` na montagem de dsk) dentro de `test_base_of_reduce_preferred`.
Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Parameterized Test Enhancement (Parametrização Direta de Dados em vez de Flag Booleana) e Declaração Explícita do Grafo. Substituí `@pytest.mark.parametrize("data_root", [True, False])` por `@pytest.mark.parametrize("c_val", [1, (f, 1)])` e declarei explicitamente os nós do grafo dsk com `c: c_val`, eliminando o `if/else` e os laços sem alterar a topologia testada.

29. test_delayed.py — Conditional Test Logic (Linha 740)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método `test_finalize_name` do arquivo `test_delayed.py` (originalmente na linha 740: `if isinstance(s, tuple): s = s[0]` e compreensão geradora na linha 745).
Minhas principais dificuldades na remoção do code smell são: O grafo resultante de `v = delayed([x])` (onde `x = da.ones(10, chunks=5)`) combina dois tipos heterogêneos de chaves no mesmo dicionário `v.dask`: chaves em formato de tupla vindas do dask.array (`('ones_like-...', 0)`) e chaves em formato str geradas pelo delayed (`'list-...'`). Por isso, a função interna `key(s)` usava um `if isinstance(s, tuple):` para normalizá-las.
Usei o CATSPY para: Identificar o Conditional Test Logic na linha 740 e a compreensão com `for` na linha 745 dentro de `test_finalize_name`.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Separação de Domínios de Chaves por Operações de Conjunto e Mapeamento Funcional. Como `x.dask` já contém exatamente as chaves em tupla e `set(v.dask) - set(x.dask)` isola exatamente as novas chaves string criadas por delayed, extraí o prefixo das tuplas com `map(lambda k: k[0], x.dask)` e validei ambos os grupos diretamente com `all(map(is_valid_key, ...))`, eliminando o `if isinstance` e o laço `for`.

30. test_cupy_sparse.py — Conditional Test Logic (Linha 83)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic no método `test_sparse_dot` do arquivo `test_cupy_sparse.py` (originalmente nas linhas 83–86: `if sp_format == "csr": ... elif sp_format == "csc": ...`).

Minhas principais dificuldades na remoção do code smell são: Não era possível referenciar `cupyx.scipy.sparse.csr_matrix` diretamente no decorador `@pytest.mark.parametrize` no nível do módulo porque cupyx é uma dependência opcional de GPU verificada apenas dentro do teste via `pytest.importorskip("cupyx")`.
Usei o CATSPY para: Detectar o Conditional Test Logic na linha 83 ocasionado pela seleção condicional de construtor (`if/elif`) baseada na string sp_format.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Conditional with Dictionary Lookup (Tabela de Despacho por Dicionário). Substituí a estrutura `if/elif` por um dicionário de mapeamento direto `sp_matrices = {"csr": cupyx.scipy.sparse.csr_matrix, "csc": cupyx.scipy.sparse.csc_matrix}` logo após o importorskip, obtendo `sp_matrix = sp_matrices[sp_format]` de forma limpa e linear.

31. test_dot.py — Exception Handling (Linha 239)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling no método `test_dot_graph` do arquivo `test_dot.py` (originalmente nas linhas 239–246: bloco `try: ... finally: ensure_not_exists(target)`).
Minhas principais dificuldades na remoção do code smell são: O teste valida proteção contra shell injection ao invocar o utilitário dot do Graphviz usando um nome de arquivo malicioso (`$(touch should_not_get_created.txt)`). Ele utilizava a fixture legada tmpdir e um bloco try/finally manual com `ensure_not_exists(target)` para limpar os artefatos gerados, além de verificar no diretório de trabalho atual se `should_not_get_created.txt` não foi criado.
Usei o CATSPY para: Localizar o smell Exception Handling na linha 239 causado pelo gerenciamento manual de limpeza de arquivos via try/finally no corpo do teste.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Try-Finally Cleanup with Pytest Fixtures (tmp_path e monkeypatch). Migrei o teste de tmpdir para tmp_path combinado com `monkeypatch.chdir(tmp_path)`, isolando tanto o arquivo de saída quanto eventuais efeitos colaterais no diretório temporário gerenciado automaticamente pelo Pytest e eliminando o try/finally e as chamadas manuais a ensure_not_exists.

32. test_config.py — Exception Handling (Linha 408)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling no método `test_ensure_file_defaults_to_DASK_CONFIG_directory` do arquivo `test_config.py` (originalmente nas linhas 408–412: `try: dask.config.PATH = destination ... finally: dask.config.PATH = PATH`).
Minhas principais dificuldades na remoção do code smell são: O teste altera temporariamente a variável global `dask.config.PATH` para um subdiretório temporário ainda inexistente (`tmp_path / "dask"`) a fim de testar a criação automática do diretório padrão por ensure_file. Caso `dask.config.PATH` não fosse restaurado corretamente em caso de falha, todos os testes subsequentes de configuração seriam afetados.
Usei o CATSPY para: Identificar o Exception Handling na linha 408 decorrente do padrão manual de salvar e restaurar estado global com try/finally.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Try-Finally Teardown with Fixture (monkeypatch). Injetei a fixture monkeypatch no teste e substituí a manipulação manual e o bloco try/finally por `monkeypatch.setattr(dask.config, "PATH", destination)`, garantindo restauração automática pelo Pytest e simplificando também a asserção final (`assert os.listdir(destination) == [source.name]`).

33. test_local.py — Exception Handling (Linha 151)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Exception Handling no método `test_exceptions_propagate` do arquivo `test_local.py` (originalmente nas linhas 151–158: `try: get({"x": (f,)}, "x"); assert False except MyException as e: ...`).
Minhas principais dificuldades na remoção do code smell são: O teste cria uma subclasse customizada de exceção `MyException(a, b)` com __str__ próprio e não apenas verifica a mensagem textual da exceção propagada pelo escalonador `dask.threaded.get`, como também inspeciona os atributos de instância `e.a == 1` e `e.b == 2` preservados após a propagação entre threads.
Usei o CATSPY para: Detectar o smell Exception Handling na linha 151 (uso do antipadrão try: ... assert False except ...) e verificar a eliminação completa do bloco try/except.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Replace Try-Except-Assert-False with pytest.raises Context Manager (exc_info). Substituí o bloco try/except por `with pytest.raises(MyException, match="My Exception!") as exc_info:`, validando a mensagem diretamente pelo parâmetro match e inspecionando os atributos preservados por meio de `exc_info.value.a == 1` e `exc_info.value.b == 2`.

34. test_expr.py — Exception Handling (Linha 155)

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Exception Handling no método `test_prohibit_reuse` do arquivo `test_expr.py` (originalmente nas linhas 155–167: `ProhibitReuse._ALLOWED_TYPES.append(FooExpr)` seguido de `try: ... finally: ProhibitReuse._ALLOWED_TYPES.remove(FooExpr)`).

Minhas principais dificuldades na remoção do code smell são:
O atributo de classe `ProhibitReuse._ALLOWED_TYPES` é uma lista global mutável compartilhada pelas expressões HLG do Dask. Para registrar temporariamente a classe de teste FooExpr sem usar try/finally, era preciso evitar mutar a lista original in-place (pois append sem remove vazaria estado global para outros testes).

Usei o CATSPY para:
Localizar o smell Exception Handling na linha 155 causado pelo bloco try/finally utilizado para desfazer a mutação na lista _ALLOWED_TYPES.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:
Replace Try-Finally State Mutation with monkeypatch.setattr (Cópia Temporária). Utilizei a fixture monkeypatch com `monkeypatch.setattr(ProhibitReuse, "_ALLOWED_TYPES", ProhibitReuse._ALLOWED_TYPES + [FooExpr])`, substituindo a referência do atributo por uma nova lista acrescida de FooExpr durante o teste e restaurando a lista original intacta automaticamente no teardown.

35. test_config.py — Non-Functional Statement (Linha 381)

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
Non-Functional Statement no método `test_set_hard_to_copyables` do arquivo `test_config.py` (originalmente na linha 381: instrução `pass` dentro de `with dask.config.set(y=1):`, além da ausência de asserções — Unknown Test na linha 376).

Minhas principais dificuldades na remoção do code smell são:

O teste foi criado originalmente como um smoke test para garantir que `dask.config.set` aninhado não quebrasse ao tentar fazer copy.deepcopy de objetos não-copiáveis como threading.Lock(). Por ter apenas pass e nenhuma asserção, ele apresentava simultaneamente Non-Functional Statement e Unknown Test.
Usei o CATSPY para: Identificar a instrução não funcional pass na linha 381 (e a falta de asserção explícita no método) e confirmar a remoção de ambos os smells após a refatoração.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Replace Non-Functional pass with Explicit State Assertions. Armazenei a instância `lock = threading.Lock()` antes dos blocos with e substituí o pass pelas asserções explícitas `assert dask.config.get("x") is lock` e `assert dask.config.get("y") == 1`, comprovando que o Lock original foi preservado por identidade na configuração aninhada.

36. test_core.py — Non-Functional Statement (Linha 39)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non-Functional Statement no método `test_ishashable` do arquivo `test_core.py` (originalmente na linha 39: instrução `pass` na definição `class C: pass`).
Minhas principais dificuldades na remoção do code smell são: A ferramenta de análise AST identifica declarações pass vazias dentro do corpo de métodos de teste como Non-Functional Statement. Era necessário instanciar uma classe customizada simples com hash padrão baseado em identidade (object.__hash__) sem manter uma instrução pass inerte dentro da função de teste.
Usei o CATSPY para: Localizar o Non-Functional Statement na linha 39 associado ao bloco vazio class C: pass dentro de test_ishashable.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Dynamic Type Construction (type()). Substituí a declaração local class C: pass pela criação dinâmica equivalente `C = type("C", (), {})`, eliminando a instrução pass não funcional e mantendo exatamente o mesmo comportamento de ishashable(C()).

37. test_delayed.py — Non-Functional Statement (Linha 74)

Eu estou atualmente trabalhando na refatoração do seguinte code smell:

Non-Functional Statement no método `test_to_task_dask` do arquivo `test_delayed.py` (originalmente na linha 74: instrução `pass` em `class MyClass(dict): pass`).
Minhas principais dificuldades na remoção do code smell são: O trecho reproduz a regressão da issue #2107, na qual subclasses customizadas de dict passadas para to_task_dask não devem ser convertidas em dict puro, devendo preservar seu tipo exato (assert type(task) is MyClass). Era preciso remover o pass interno sem perder a herança direta de dict.
Usei o CATSPY para: Detectar o Non-Functional Statement na linha 74 dentro de test_to_task_dask e verificar sua eliminação pós-refatoração.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Dynamic Subclass Construction (type()). Substituí class MyClass(dict): pass por `MyClass = type("MyClass", (dict,), {})`, criando a subclasse de dict em uma única expressão funcional sem instrução pass.

38. test_utils.py — Non-Functional Statement (Linha 524)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non-Functional Statement no método `test_ensure_set` do arquivo `test_utils.py` (originalmente na linha 524: instrução `pass` em `class myset(set): pass`).
Minhas principais dificuldades na remoção do code smell são: O teste verifica se ensure_set converte instâncias de subclasses de set (myset(s)) de volta para o tipo nativo set (assert type(si) is set). Assim como nos casos anteriores, a definição de classe vazia dentro do método gerava uma instrução pass classificada como Non-Functional Statement.
Usei o CATSPY para: Identificar a instrução não funcional na linha 524 de test_ensure_set e confirmar a limpeza do smell.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Dynamic Subclass Construction (type()). Substituí a declaração class myset(set): pass por `myset = type("myset", (set,), {})`, preservando integralmente a verificação de conversão de subclasse para set sem instrução pass.

39. test_array_core.py — Non-Functional Statement (Linha 3033)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Non-Functional Statement no método `test_from_array_inline` do arquivo `test_array_core.py` (originalmente na linha 3033: instrução `pass` em `class MyArray(np.ndarray): pass`).
Minhas principais dificuldades na remoção do code smell são: O teste utiliza .view(MyArray) sobre um np.ndarray para validar que da.from_array(..., inline_array=False) armazena a referência em dsk["original-my-array"] respeitando subclasses de np.ndarray. Era necessário garantir que a classe criada dinamicamente fosse totalmente compatível com ndarray.view(...) do NumPy.
Usei o CATSPY para: Localizar o Non-Functional Statement na linha 3033 dentro de test_from_array_inline e validar sua remoção.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Dynamic Subclass Construction (type()). Substituí class MyArray(np.ndarray): pass por `MyArray = type("MyArray", (np.ndarray,), {})`, mantendo compatibilidade total com np.array([1, 2, 3]).view(MyArray) e eliminando a instrução pass.

40. test_collection.py (dask_expr) — Redundant Print (Linha 834)

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Redundant Print no método `test_to_datetime` do arquivo `test_collection.py` (originalmente na linha 834: `print(expected)`).
Minhas principais dificuldades na remoção do code smell são: Confirmar se a chamada print(expected) (onde expected = pd.to_datetime(ps.index)) exercia qualquer efeito colateral necessário na avaliação do DatetimeIndex antes de assert_eq(result, expected, check_divisions=False). Como expected é um objeto Pandas já materializado em memória e validado logo na linha seguinte por assert_eq, tratava-se apenas de saída residual de depuração poluindo o stdout da suíte.
Usei o CATSPY para: Detectar o smell Redundant Print na linha 834 do método test_to_datetime e confirmar sua remoção.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Remove Redundant Print (Remoção de Instrução de Depuração). Removi a linha print(expected) do corpo de test_to_datetime, mantendo todas as verificações assert_eq e pytest.raises(NotImplementedError) intactas.